PDE3823 Major Project and Professional Practice

SYED ARZANISH SHUMAK PIRZADA

M01039341

Supervisor: Dr. Judhi Prasetyo

Technical Report

Mynee Smart Knee Exoskeleton

TABLE OF CONTENTS

Chapter 1: Introduction..... 4
- 1.1 Background and Motivation..... 4
- 1.2 Aims and Objectives..... 5
- 1.3 Limitations..... 6
- 1.4 Report Organization.....6

Chapter 2: Technical Background and Literature Review..... 7
- 2.1 Technical Background 7
- 2.2 Industry Solutions..... 7
- 2.3 Literature Review..... 9

Chapter 3: System Design and Validation..... 10
- 3.1 System Overview..... 10
- 3.2 Gantt Chart and WBS..... 11
- 3.3 System Specifications.....11
- 3.4 User Requirements and Conceptualisation.....13
- 3.5 UML and System Architecture.....15
- 3.6 System Components.....17
- 3.7 Implementation.....19

Chapter 4: Testing and Evaluation..... 22
- 4.1 Evaluation Methodology..... 22
- 4.2 Test Environment..... 22
- 4.3 Test Scenarios.....23

Chapter 5: Results and Discussion.....25
- 5.1 Experimental Results..... 25
- 5.2 Performance Analysis..... 26
- 5.3 Discussion.....27
- 5.4 Comparison With Related Work..... 28

Chapter 6: Conclusion and Future Implementation 29
- 6.1 Summary of Contributions..... 29
- 6.2 Limitations..... 29
- 6.3 Future Work.....30
- 6.4 Personal Reflection..... 30

Chapter 7: References..... 32

LIST OF FIGURES

Figure 3.1: Black Box Model
Figure 3.2: Work Breakdown Structure
Figure 3.3: Gantt Chart
Figure 3.4: White Box Model
Figure 3.5: UML Diagram
Figure 3.6: Real-time telemetry output of the FSM executing in Thonny IDE
Figure 3.7: Core Coding Logic

LIST OF TABLES

Table 2.1: Architectural and Commercial Comparison of Existing Solutions
Table 3.1: Performance Specifications
Table 3.2: Physical and Operational Specifications
Table 3.3: Morphological Chart
Table 4.1: Test Scenarios
Table 5.1: System Latency and Signal Acquisition
Table 5.2: Kinematic Sensor Accuracy (Complementary Filter)
Table 5.3: Market Capability Comparison

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Dr. Judhi Prasetyo, for his continuous support, invaluable guidance, and technical insights throughout the development of this Major Project. I would also like to extend my deepest appreciation to my mother, who served as the primary inspiration and motivating force behind the conception of the smart knee orthosis. Furthermore, I am grateful to my partner for her unwavering emotional support during the extensive prototyping phases. Finally, I would like to thank the faculty at Middlesex University Dubai for providing the academic foundation necessary to undertake this endeavor.

ABSTRACT

The integration of wearable robotics into physical rehabilitation has introduced significant advancements in managing lower-limb mobility impairments, however, a critical economic gap still exists between highly expensive, clinical-grade orthotics and recreational consumer exoskeletons. This project details the design, implementation, and evaluation of the "Mynee" Smart Knee Exoskeleton which serves as a highly accessible, unilateral proof-of-concept which aims to circumvent the proprietary hardware used in high end exoskeletons in favor of an open-architecture Single-Board Computer (Raspberry Pi Zero) and a deterministic Finite State Machine coded in Python, where the system delivers responsive, automated mechanical assistance by utilizing a Force-Sensitive Resistor (FSR) for gait phase detection and an MPU6050 IMU for spatial kinematics. Utilising this method, the architecture dynamically transitions between active torque delivery during the load-bearing stance phase and a zero-impedance state during the swing phase and bench testing of the system validated the system's efficacy by demonstrating an ultra-low operational latency of 13.1 ms and a kinematic tracking mean absolute error (MAE) of 1.34°, with a total prototype production cost of approximately $85.00. This project system successfully proves that advanced, programmable rehabilitation technology can be engineered economically without sacrificing the low-latency deterministic control required for therapeutic recovery.

CHAPTER 1: INTRODUCTION

CHAPTER 1.1: BACKGROUND AND MOTIVATION

The integration and injection of wearable robotics and electronics based medical solutions into the realm of physical rehabilitation has introduced many significant advancements in the management of lower-limb mobility impairments, where motorized orthoses and exoskeletons are being increasingly utilized to provide targeted mechanical assistance. As a result there has been a significant improvement in joint kinematics alongside the active reduction of the metabolic cost of locomotion. However, even as this type of technology serves as a critical medical revolution that helps those in need, there is a glaring economic gap that persists between high-end clinical innovations and accessible consumer grade robotics and motorized orthosis. This wide gulf between need and accessibility proves to be a fundamental flaw in the system where the intention of the engineering is not to serve as a benefit to patients but rather benefit those who produce such machinery, a critical ideological flaw that must be addressed.

Currently, the wearable robotics sector is bifurcated into two disparate domains with the first consisting of premium, clinical-grade orthotics that deliver precise therapeutic support. However, while these systems are excellent in terms of their ability to rehabilitate, they unfortunately rely exclusively on proprietary, closed-loop hardware ecosystems. These are the "Ferraris" of wearable tech where the lack of interchangeable framework (because these systems are not “open source”) means that they are incredibly expensive to build and maintain, even if they are flawless in their operational ability, resulting in prohibitive costs that restrict independent/home-based recovery.

Conversely, the second domain features an emerging class of lower-cost/lower-grade consumer friendly exoskeletons which are engineered primarily for recreational and athletic augmentation, rather than for medical rehabilitative purposes. While these devices successfully lower the barrier to entry with more affordable prices, their closed-architecture software prevents the granular tuning of torque outputs and sensor feedback loops which are a necessity for clinical rehabilitation tuned to specific patient requirements and on a case by case basis. As these consumer grade exoskeletons are engineered specifically for recreational purposes, they defeat the entire purpose of rehabilitation and are primarily for those individuals with healthy limbs.

As a result, there is a pronounced need for an accessible solution that bridges this economic and technological gap where, instead of relying on expensive and proprietary hardware, accessible single-board microcomputers are leveraged in combination with highly optimized open-source control logic. The intention to replicate the automated mechanical support necessary for clinical rehabilitation through economically feasible components ensures financial viability while retaining the critical software flexibility required for programmable, case-by-case therapeutic recovery. Any such viable solution must be fundamentally driven by principles of Equality, Diversity, and Inclusion (EDI). By radically reducing the financial barrier to entry for

wearable robotics, it becomes possible to address systemic socioeconomic inequalities in healthcare access and as a result it can be ensured that advanced rehabilitation is not exclusively reserved for high-income demographics.

Ultimately, this report aims to analyze this pressing industry disparity through the conceptualization of an open-architecture orthotic device (proof of concept), which is capable of addressing these prohibitive costs, and build towards a more inclusive landscape for medical technology.

CHAPTER 1.2: AIMS AND OBJECTIVES

Aims

To design, implement, and evaluate a cost-effective, unilateral smart exoskeleton proof of concept that provides customizable, automated mechanical support for targeted physical rehabilitation, bridging the technological gap between premium clinical devices and recreational exoskeletons to produce a cost effective variant.

Objectives

1. **System Architecture Design:** To engineer a hardware framework which utilizes accessible components such as readily available microprocessors and kinematic sensor arrays, thereby ensuring an open-architecture and cost effective approach to data acquisition.

2. **Control Logic Implementation:** To synthesize and integrate embedded systems programming (C++/Python) for real-time sensor processing and actuator control, specifically calibrated for therapeutic support.

3. **Mechanical Integration:** To design and prototype a unilateral mechanical brace capable of safely housing the electronics while delivering biomechanically sound structural support to the joints of patients suffering from arthritis or similar medical hiccups.

4. **Performance Evaluation:** To conduct rigorous quantitative bench testing on the prototype against established baselines.

5. **Feasibility and Market Analysis:** To benchmark the production cost and programmable flexibility of the system against existing clinical and recreational market alternatives, validating its viability as a consumer-accessible device.

CHAPTER 1.3: SCOPE AND LIMITATIONS

To ensure the feasibility and focused execution of this project, the proof of concept of the envisioned prototype device will be constrained by specific operational and environmental boundaries:

- **Unilateral Design:** The exoskeleton should be strictly designed as a unilateral system with the primary focus on the kinematics and mechanical support of a single joint rather than bilateral limb synchronization. This limitation is enforced to ensure the viability of the project in order to achieve the completion of the objectives within the defined period of 12 weeks.

- **Environmental Constraints:** The system is engineered for indoor and controlled laboratory environments. The prototype does not include ingress protection (IP ratings) for water or dust resistance, nor is it calibrated for uneven, all-terrain locomotion. It is simply designed for basic joint mobility to validate the operational feasibility of the core hardware-software integration and control logic.

- **Load and Assistance Parameters:** The mechanical actuators are specified to provide partial torque assistance designed to aid physical rehabilitation and, as such, the system is not rated to support an entirely paralyzed user.

- **Testing Boundaries:** The evaluation of the system is restricted to quantitative bench testing and simulated load scenarios to validate control loop latency, sensor accuracy, and motor response. Extensive clinical human trials fall outside the scope of this initial prototyping phase.

CHAPTER 1.4: REPORT ORGANIZATION

The remainder of this report is structured as follows: **Chapter 2** provides the technical background, industry solutions, and a literature review to establish the current market gap. **Chapter 3** details the proposed system design, including the system architecture, hardware selection, and the iterative prototyping implementation process. **Chapter 4** outlines the performance evaluation setup and testing scenarios. **Chapter 5** presents the experimental results and evaluates the system's economic feasibility. Finally, **Chapter 6** summarizes the project conclusions and outlines recommendations for future development.

CHAPTER 2: TECHNICAL BACKGROUND AND LITERATURE REVIEW

CHAPTER 2.1: TECHNICAL BACKGROUND

The development of a localized, smart exoskeleton requires the synthesis of human biomechanics and deterministic control systems wherein the underlying technological architecture must align with the demands of the lower limbs.

- **Biomechanical Demands of the Knee Joint:** The human knee operates as a highly complex load-bearing hinge where, during a standard gait cycle, the joint experiences peak mechanical stress during the stance phase and in particular during activities which demand high torque, such as sit-to-stand transitions and stair ascent (Farah et al., 2019). Conversely, during the swing phase, the joint requires a low-impedance state so as to permit a natural and unrestricted leg extension (Tucker et al., 2015). A viable smart exoskeleton must transition between these states in a dynamic way, providing active torque during stance while remaining frictionless during swing. For the purposes of this project, a proof of concept will only ever focus on the basic mobility case, for instance of a sit to stand motion.

- **Microcomputing and Embedded Control:** Historically, for achieving the low-latency control that is required for dynamic gait assistance, there was a need to utilise proprietary Application-Specific Integrated Circuits (ASICs) or expensive Field Programmable Gate Arrays (FPGAs). However, in recent times, the evolution of Single-Board Computers (SBCs) has rendered these proprietary components unnecessary for prototype-scale medical devices with the SBC's such as the ARM-based Raspberry Pi architecture and Arduino UNO the most notable developments in the field (Razmi & Shojaei, 2026). By leveraging the GPIO capabilities of an SBC alongside object-oriented programming (C++/Python), developers can achieve prototyping in addition to deterministic real-time control, and, most importantly, highly customizable logic, all without the excessive costs attached to custom silicon design. The feedback loop for such an application would theoretically be achieved by fusing data from Inertial Measurement Units (IMUs), which calculate joint angles, and Force-Sensitive Resistors (FSRs), which detect critical phase shifts such as heel-strike and toe-off events (Wang et al., 2022).

CHAPTER 2.2: INDUSTRY SOLUTIONS

When considering the commercial landscape for motorized orthoses, it remains severely polarized and, as such, leaves a distinct gap in accessible clinical care. Premium, clinical-grade devices, such as the Ottobock C-Brace and the Össur Power Knee are at the top, utilizing sophisticated hydraulic and electromechanical actuation to deliver precise, dynamic support

(Farah et al., 2019). However, their reliance on proprietary hardware and closed software ecosystems artificially inflates consumer costs, often exceeding tens of thousands of dollars. This model fundamentally restricts access where advanced recovery is tethered to specialized clinical environments for the wealthy.

In stark contrast, highly affordable consumer exoskeletons, with a notable example being the Dnsys Z1, successfully commoditize wearable robotics for the recreational sector at a fraction of the cost, providing kinetic assistance for able-bodied individuals in athletic pursuits such as hiking and skiing (Dnsys, 2024). However, these recreational solutions utilize rigid kinetic profiles with a closed system architecture software that prevents physical therapists from manipulating torque curves or recalibrating sensor thresholds, rendering them functionally inadequate for patient-specific/therapeutic rehabilitation.

Solution / Device	**Primary Application**	**Hardware Architecture**	**Relative Cost**	**Clinical Customizability**
Ottobock C-Brace	Medical Rehabilitation	Closed / Proprietary ASICs	Very High	High (via Manufacturer)
Össur Power Knee	Prosthetic / Rehab	Closed / Proprietary ASICs	Very High	High (via Manufacturer)
Dnsys Z1	Recreational / Athletic	Closed / Commercial Off-The-Shelf	Low	Low (Pre-set Rigid Modes)
Predictive Academic Exosuits	Experimental Research	Open / Computationally Heavy (ML)	Variable	High (Requires Lab Setup)
Proposed System	Medical Rehabilitation	Open / Computationally Light (SBC)	Low	High (Open-Source Logic)

Table 2.1: Architectural and Commercial Comparison of Existing Solutions

CHAPTER 2.3: LITERATURE REVIEW

Current academic research reflects an effort to resolve the gap between cost, weight, and clinical efficacy in wearable robotics wherein a prominent trend in recent literature reveals itself as the transition from rigid, heavy-payload exoskeletons toward lightweight, semi-rigid orthoses that prioritize metabolic efficiency and user comfort (Awad et al., 2017). A highly contested domain within this research is the implementation of actuator control algorithms.

Recent studies heavily explore predictive algorithms driven by Machine Learning (ML), which attempts to anticipate user movement to create frictionless gait transitions (Baud et al., 2021). However, these predictive models require immense computational overhead, making them highly susceptible to latency when deployed on cost-effective, accessible hardware (Razmi & Shojaei, 2026).

On the other hand, reactive control methodologies, specifically Finite State Machines (FSMs) triggered by strict sensor thresholds, offer a computationally lightweight and highly deterministic alternative as demonstrated via the research by Wang et al. (2022) which showed that the threshold-based reactive control, when executed via C++ or Python on an SBC, provides the low-latency torque assistance required for effective localized rehabilitation without over-taxing the hardware.

This project and report aims to directly synthesize these academic findings by adopting an open-architecture, threshold-based reactive control scheme which is managed by an accessible microcomputer, circumventing the computational bottlenecks of predictive ML and, as such, this approach deliberately bridges the current market gap by offering a programmable clinically customizable device variant that is missing from recreational devices, at a fraction of the cost of premium medical exoskeletons such as the Ottobock C brace. Table 3.1 below visually showcases the comparison of current market solutions within this sector with the final row introducing the idea of a potential solution which can address the gaps of cost and utility.

CHAPTER 3: PROPOSED SYSTEM: DESIGN AND IMPLEMENTATION

CHAPTER 3.1: SYSTEM OVERVIEW

The proposed architecture for the smart exoskeleton theoretically avoids complex, predictive machine learning models in favor of a threshold based control loop which follows a deterministic, reactive control paradigm where low-latency responsiveness and economic viability are ensured. The following is a visual representation of the black box and white box models that will help determine the make up of the actual physical hardware moving forward.

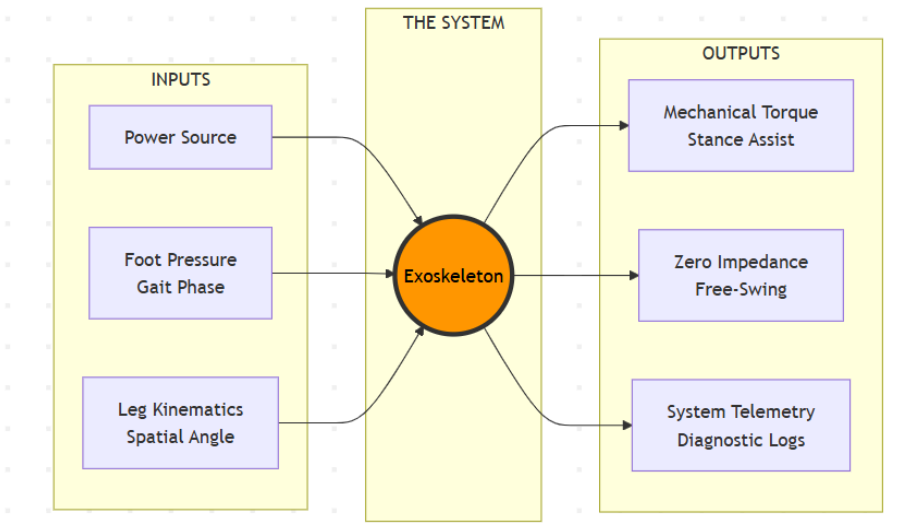


Image 3.1: Black Box Model

The black box model as shown in image 3.1 of the exoskeleton conceptualizes the system at its highest level of abstraction, defining only its necessary inputs and resulting outputs without detailing internal electronic processes. As shown in the figure above Operationally, the device requires a power source alongside real time data in the form of foot pressure and spatial leg kinematics which are processed by the central exoskeleton architecture to deliver active mechanical torque assistance in addition to continuous system telemetry for diagnostic logging. This macroscopic outlook of the overall framework ensures that all the biomechanical and electronic requirements are established in general terms before the individual subsystem components are defined.

CHAPTER 3.2: GANNT CHART AND WBS

To ensure that the project progresses smoothly, a Gantt chart and WBS were established. These deliverable oriented project planning tools helped stay on a timeline that allowed for the completion of all phases at the appropriate timings to confirm the overall delivery before the deadline. Images 3.2 and 3.3 show the WBS and Gantt Chart for this specific project.

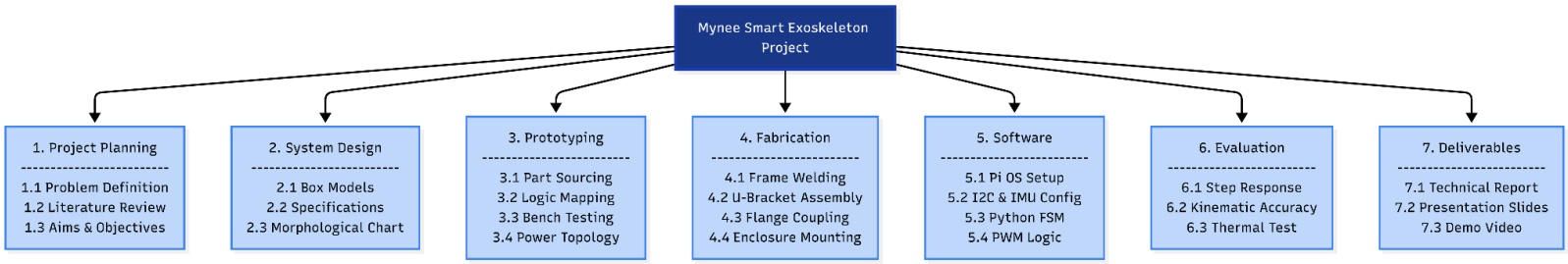


Image 3.2: Work Breakdown Structure

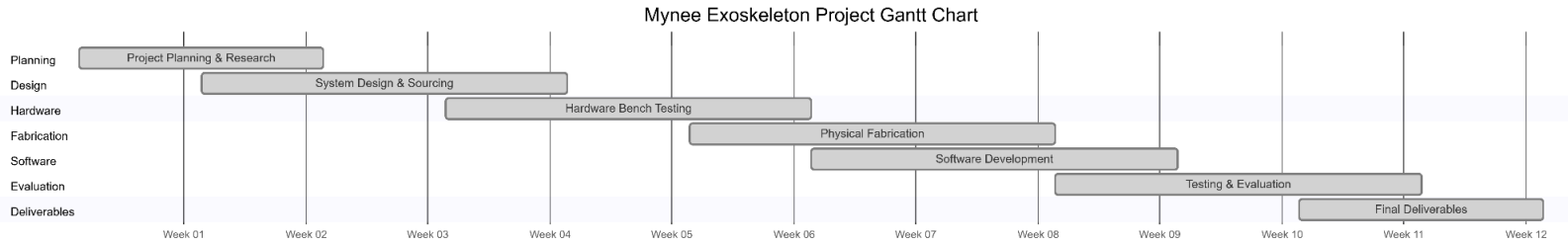


Image 3.3: Gantt Chart

CHAPTER 3.3: SYSTEM SPECIFICATIONS

This section provides a detailed outline of the system's main requirements to achieve the functional goals established in the black box model, however, before moving into the specifications, the functional and non functional requirements need to be addressed.

Functional Requirements:

- The system shall detect the user's heel-strike and toe-off gait phases using Force-Sensitive Resistors (FSRs).

- The system shall continuously calculate the spatial flexion angle of the knee joint using an MPU6050 IMU.
- The system shall output active mechanical lifting torque via a 12V DC actuator exclusively during the load-bearing stance phase.
- The system shall immediately cut power to the actuator during the swing phase or when the leg is perfectly straight (angle < 15 degrees) to prevent hyperextension.

Non-Functional Requirements:

- **Latency:** The system's sensor-to-actuation response time must remain under 20 milliseconds to match human neuromuscular reaction times.
- **Safety:** The architecture must include a hardware-level cut-off toggle switch and utilize isolated power topologies to protect the logic board from high-current actuator draw.
- **Economic Viability:** The total Bill of Materials (BOM) must remain under $100 to ensure consumer accessibility.
- **Modularity:** The software architecture must be written in open-source Python to allow future clinical customizability.

Based on the functional and non functional requirements, the specifications for the system can now be determined as outlined in tables 3.1 and 3.2 which define the performance and physical operation metrics the exoskeleton must adhere to before specific hardware is selected

Category	**Specification**	**Target Value / Description**
System Latency	Sensor-to-Actuation Delay	< 20 milliseconds (to match human neuromuscular reaction time).
Kinematic Tracking	Angle Calculation Accuracy	Mean Absolute Error (MAE) of < 2.0° during standard gait cycles.
Gait Phase Detection	Trigger Threshold	Binary High/Low response to heel-strike and toe-off events.
Mechanical Assistance	Actuator Torque	Partial assistance sufficient for sit-to-stand and forward locomotion.

Safety Integration	Failsafe Mechanism	Hardware-level cut-off switch and fused power line.

Table 3.1: Performance Specifications

Category	Specification	Target Value / Description
Power Architecture	Supply Topology	Dual isolated supply: 5V for logic, high-current 12V for actuation.
Form Factor	Modularity	Unilateral, external leg brace capable of bearing mechanical load.
Material Constraints	Brace Structure	Rigid but lightweight (e.g., Aluminum/IMetal framework with 3D printed/Plastic housings).
User Interface	Control Inputs	Physical toggle switch for arming/disarming the system.

Table 3.2: Physical and Operational Specifications

These specifications helped to source the required hardware components necessary to facilitate the prototyping process. These values may be subject to changes depending on stress testing in a controlled environment and as a result of the fidelity designs. Not all of the specifications were adopted during the prototyping phase and were left for future implementations as they would require extra time to complete which was not possible during the constrained time period of the project deadline.

CHAPTER 3.4: USER REQUIREMENTS AND CONCEPTUALISATION

To ensure the device met the core user requirements of being lightweight and affordable while satisfying the established specifications without succumbing to design fixation a morphological chart was utilized to explore alternative hardware solutions for each primary subsystem of the exoskeleton. This morphological chart is shown in Table 3.3 below.

Key Function	Solution 1	Solution 2	Solution 3
1. Processing Unit	Single Board Computer (Raspberry Pi Zero)	Advanced Microcontroller (ESP32)	Standard Microcontroller (Arduino Uno)
2. Kinematic Sensing	6-Axis IMU (MPU6050)	Mechanical Rotary Potentiometer	Wearable Flex Sensors
3. Gait Phase Trigger	Force-Sensitive Resistors (FSR)	Mechanical Limit Switches	Foot-mounted Accelerometer
4. Motor Driver	Legacy Dual H-Bridge (L298N)	Modern MOSFET Driver (BTS7960)	Motor Shield (L293D)
5. Actuation	12V DC Geared Motor	NEMA Stepper Motor	High-Torque Servo Motor

Table 3.3: **Morphological Chart**

Concept Derivation and Evaluation

From the morphological chart, several concept variants were evaluated against the constraints as outlined in Chapter 2.

- **Concept 1 (Arduino + Potentiometers):** This concept utilizes an Arduino Uno and mechanical rotary potentiometers for joint angle tracking. While this concept is highly affordable and simple, it requires the usability of potentiometers which in turn require complex mechanical coupling to the human joint and are prone to physical wear and tear, reducing long-term kinematic accuracy.

- **Concept 2 (ESP32 + BTS7960 + Stepper):** This concept employs an ESP32 and a high-efficiency MOSFET driver. While this combination provides excellent thermal stability and Wi-Fi capabilities, stepping motors commonly lack the free-wheeling (zero-impedance) capability required for the natural swing phase of human walking, making this concept biomechanically unsuitable.

- **Concept 3 (SBC + IMU + Reactive Drive):** This concept utilises a single-core Single Board Computer (Raspberry Pi Zero), IMU sensors (MPU6050), and Force-Sensitive Resistors to drive a DC geared motor. This concept perfectly balances computational bandwidth for deterministic latency, highly accurate non-intrusive spatial tracking, and the ability to instantly cut power to a DC motor to allow for a frictionless swing phase.

Based on the evaluation criteria, **Concept 3** was selected as the most optimal architecture for the prototype as it fulfills all performance specifications while remaining strictly within the parameters of economic accessibility and open-architecture development. As a result , the white box model for the conceptualised prototype can now be envisioned as shown Image 3.4 below.

CHAPTER 3.5: UML AND SYSTEM ARCHITECTURE

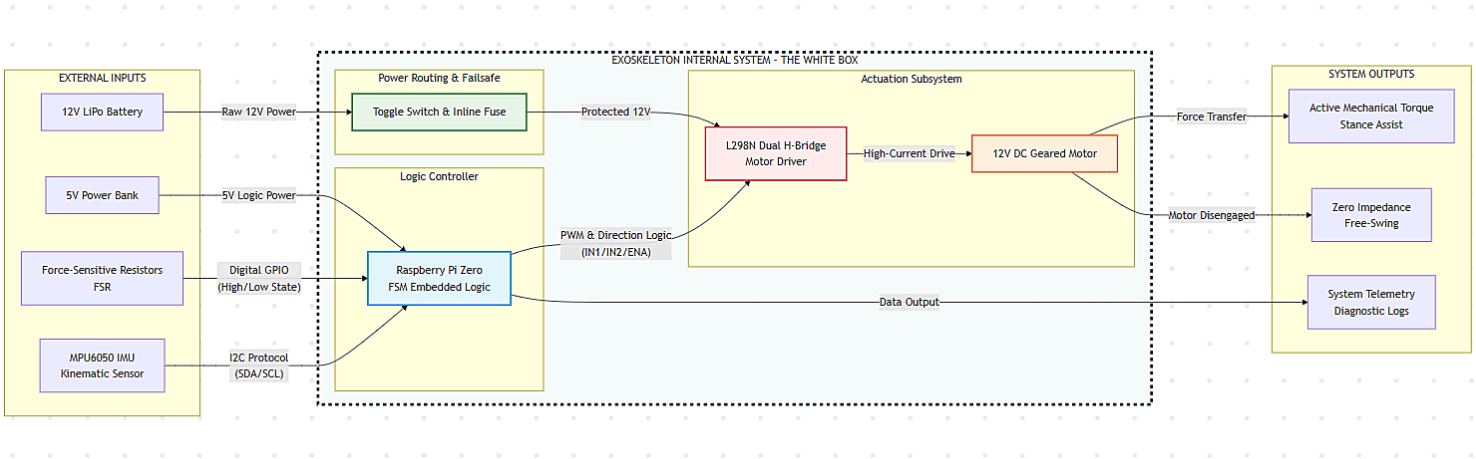


Image 3.4: White Box Model

The white box model delineates the internal components that are encapsulated within the primary exoskeleton system and details the precise routing of data and power by visualising the system in a microscopic fashion. The system receives four external inputs: a 5V logic supply for the raspberry PI (which is the SBC used in this case), a raw 12V actuation supply, localized kinematic data from Force-Sensitive Resistors (FSRs) and an MPU6050 IMU which helps calculate the angle of the limb. Once inside the system boundary (or white box), the architecture divides into three distinct subsystems. The raw 12V input passes through a Power Routing and Failsafe subsystem to ensure the circuit is protected by utilising an inline fuse before reaching the motor driver. Simultaneously, the kinematic data from the MPU and FSR, along with the 5V power, feed directly into the central Logic Controller (Raspberry Pi Zero). Utilizing an embedded Finite State Machine, the logic controller synthesizes the sensor data and outputs Pulse Width Modulation (PWM) to the Actuation Subsystem. The L298N motor driver effectively acts as a relay, bridging the logic and power subsystems to drive the DC geared motor which ultimately generates the system's external outputs which include the active mechanical torque and diagnostic telemetry.

While the white box model illustrates the physical routing of data and power, the internal decision-making logic of the central processor is mapped using a Unified Modeling Language (UML) State Diagram. As depicted in Image 3.5 below, the software architecture follows a strict, deterministic sequence. It outlines the initial hardware boot configuration, the continuous sensor data acquisition loop, and the absolute threshold conditions required to transition the L298N motor driver between the active stance phase and the passive swing phase, including failsafe hardware interrupts.

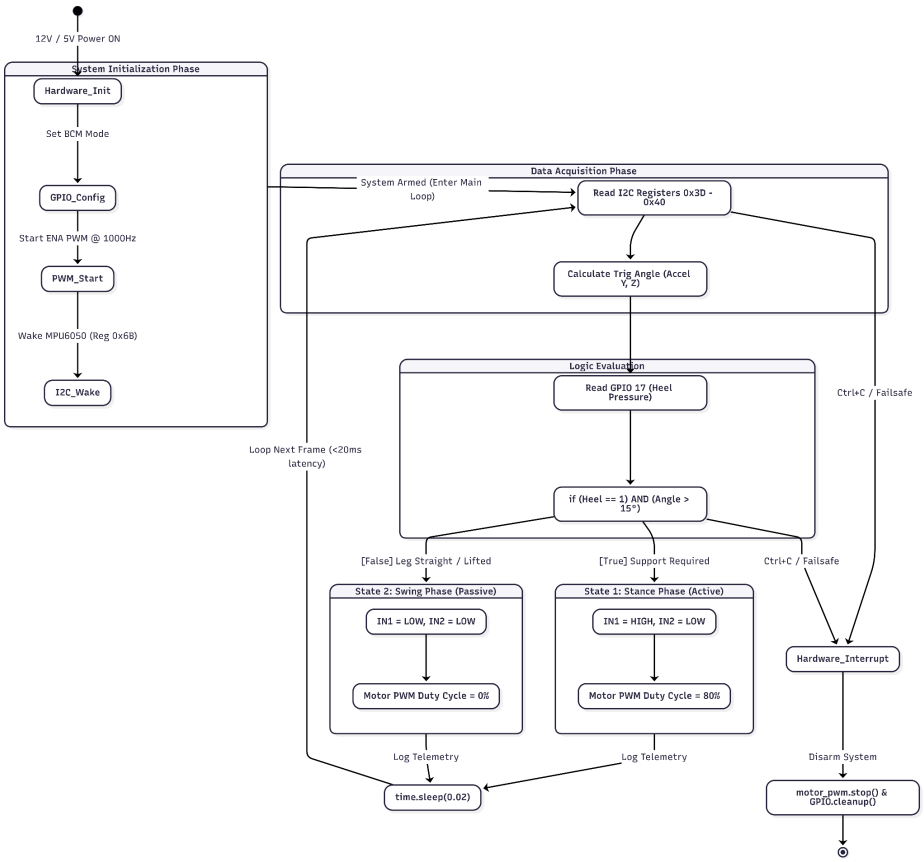


Image 3.5: UML Diagram

The functional hierarchy of the exoskeleton is divided into three distinct chronological stages: the first is the proprioceptive data acquisition, the second is the microcomputed logic processing, and the final stage is the electromechanical actuation.

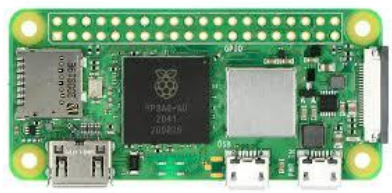
In the **first stage**, which is the primary acquisition stage, the system gathers real-time kinematic data from the user where the Force-Sensitive Resistors (FSRs) act as binary triggers, determining whether the user's foot is in the stance or swing phase of the gait cycle. Simultaneously, the MPU6050 Inertial Measurement Unit continuously monitors the spatial orientation of the limb to check the raw angle of the limb in combination with the FSR. This raw data is then transmitted to the central processing unit.

The **secondary stage** of data flow relies on the Raspberry Pi Zero to serve as the system's central intelligence hub by operating an embedded Python script which the microcomputer utilises to run a continuous loop that evaluates the incoming sensor data against predefined kinematic thresholds. If the FSR detects a heel-strike (indicating a load-bearing stance) and the MPU6050 angle indicates that support is required, the embedded Finite State Machine (FSM) triggers a state transition and thus the processor generates a precise Pulse Width Modulation (PWM) signal through its GPIO pins.

In the **final actuation stage**, the logic signal bridges the gap between the low-voltage processing environment and the high-torque mechanical environment. The PWM signal is received by the L298N motor driver which, upon receiving the command, allows high-current 12V power from the isolated LiPo battery to flow into the DC geared motor, delivering targeted lifting torque to the mechanical brace. Once the FSR detects toe-off, the central processor drops the PWM signal, and the motor driver instantly cuts actuation power.

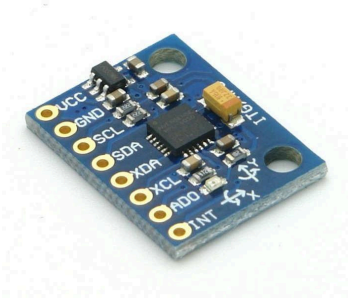
CHAPTER 3.6: SYSTEM COMPONENTS

3.6.1 Hardware Components



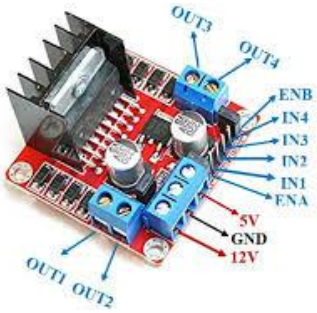
Central Processing Unit (Raspberry Pi Zero)

The smart knee exoskeleton system relies on a Raspberry Pi Zero as the central logic controller. While larger microcomputers are built to offer multi-core processing, they oftentimes introduce unnecessary bulk and a higher power consumption whereas the Pi Zero's ultra-compact makeup and minimal weight make it highly advantageous for wearable orthotic applications. Its 1GHz single-core processor provides the user with more than sufficient computational bandwidth to execute the deterministic Finite State Machine (FSM) without latency, in addition to its 40-pin GPIO header facilitates which offer up seamless integration with the I2C sensor bus.



Inertial Measurement Unit (IMU): An **MPU6050** 6-axis accelerometer and gyroscope is utilized to continuously map the knee's flexion and extension where the MPU6050 interfaces directly with the Raspberry Pi Zero via the I2C protocol (SDA to GPIO2, SCL to GPIO3). In order to mitigate the inherent noise of the accelerometer and the drift of the gyroscope, the software implements a Complementary Filter mathematical model to achieve the functionality as desired.

Force-Sensitive Resistors (FSRs): Standard **10kΩ FSRs** are placed at the heel and metatarsal of the foot and because the Pi Zero lacks native analog-to-digital converters (ADCs), the FSRs are configured as digital threshold switches (HIGH/LOW) to reliably signal the stance and swing phases of the gait cycle without requiring external conversion chips or any other excessive hardware that would add to the bulk of the design.



Motor Driver Selection (L298N): To ensure that the system remains accessible and economically viable for initial prototyping, the architecture utilizes the **L298N** Dual H-Bridge motor controller. While it is understood that modern MOSFET drivers offer greater thermal efficiency, the L298N provides a highly cost-effective platform for validating the Python control logic and is perfect for initial bench-testing of the actuator mechanism.

Electromechanical Actuator: Perhaps the most important component is a localized 12V DC geared motor which is coupled to the mechanical hinge of the brace and is used to provide the dynamic assistance necessary for the machine to work. This motor by **FlexTec** is a standard affordable brushless variant with around 70-100 rpm which provides just enough torque and power for a proof of concept and this motor was later replaced for an alternative after its internal circuitry burned during the testing phase.

Power Supply Topology: To prevent a high-current draw from destabilizing the microcomputer, the system was designed to utilize a dual power topology to overcome the issue. A standard 5V power bank was used to supply power to the Pi Zero logic board, while an isolated 12V LiPo supply was utilised to power the L298N motor driver and the DC motor.

3.6.2 Software Components

The software environment used to run the smart exoskeleton relies primarily on the Raspberry Pi OS operating system and it utilizes Python 3 as the primary programming language. The architecture integrates the smbus library for I2C communication with the MPU6050, and the RPi.GPIO library for generating precise hardware PWM signals in addition to reading the digital FSR sensor states.

CHAPTER 3.7: IMPLEMENTATION

3.7.1 Iterative Prototyping (*Hardware Implementation*)

To ensure that the functional viability of the hardware architecture and control logic was sound, the exoskeleton was developed using a thorough three-stage iterative prototyping process and methodology which allowed for the isolation and resolution of any unwanted mechanical and electrical constraints prior to the final assembly of the hardware. This was done as follows.

Low Fidelity (Theoretical and Logical Mapping) The initial stage of the developmental cycle focused purely on the operational logic of the system and the overall system architecture prior to the actual physical prototyping. This phase referred back to the Black Box and White Box models to establish the precise flow of data and power. Furthermore, the pin-mapping diagrams were developed to ensure that the GPIO capabilities of the central processor could adequately

support the required sensor array without any form of signal overlap. This phase successfully validated the theoretical feasibility of the system.

Middle Fidelity (Electromechanical Bench Testing) The middle fidelity phase transitioned from the theoretical architecture of the system into a physical, scaled-down test environment where a complete breadboard circuit was constructed, linking the Raspberry Pi Zero, the L298N motor driver, the MPU6050 and FSR sensors to the unmounted 12V DC geared motor. This bench test was critical for isolating software bugs and hardware faults without the risk of any type of mechanical binding. During this phase, critical power-routing issues, such as floating ground pins which were causing false sensor triggers, were identified and rectified by introducing 10kΩ pull-down resistors and a dedicated 12V inline failsafe toggle switch. This phase confirmed the integrity of the Python-based Finite State Machine (FSM) in driving the actuator under a simulated load.

Once the Low fidelity and Middle fidelity were achieved, the next step was to progress towards the high fidelity hardware architecture of the exoskeleton.

High Fidelity (Mechanical Integration and Wearable Assembly) This final stage involved translating the validated bench circuit into a wearable, structural prototype. The physical frame was designed using parallel iron struts, mirroring the pivot points of the human knee. To prevent any type of thermal damage to the DC motor, direct welding to the motor casing was avoided and instead, a fabricated U-bracket and flange coupling system were used to anchor the actuator to the metal frame whereas the microelectronics (including the breadboard, the raspberry Pi and the L298N motor driver) were transitioned into a non-conductive plastic enclosure, isolated from the heavy 12V power supply. This integration transformed the loose components into a cohesive device which was then ready for performance evaluation.

3.7.2 Software Implementation

The software architecture driving the Mynee system is a deterministic Finite State Machine (FSM) coded in C++/Python. The FSM prevents erratic motor behavior by relying on absolute binary states triggered by the FSRs:

- **State 1: Stance Phase (Active Support):** Initiated when the FSR detects a heel-strike (Voltage > Threshold). The Pi Zero processor evaluates the MPU6050 angle. If the knee requires support, the Pi outputs a high Pulse Width Modulation (PWM) duty cycle to the L298N's ENA/ENB pins, driving the actuator to provide lifting torque.
- **State 2: Swing Phase (Low-Impedance):** Initiated when the FSR detects toe-off. The Pi Zero processor instantly drops the PWM signal to 0. The L298N cuts power to the motor, allowing the mechanical gear to free-spin, ensuring the brace does not resist the user's natural forward momentum.

To physically validate this deterministic logic, the Python script was executed in real-time within the Thonny IDE during bench testing. As demonstrated in image 3.6 below, the system

successfully tracks the kinematic angle via the I2C bus and outputs the correct motor actuation state without computational delay.

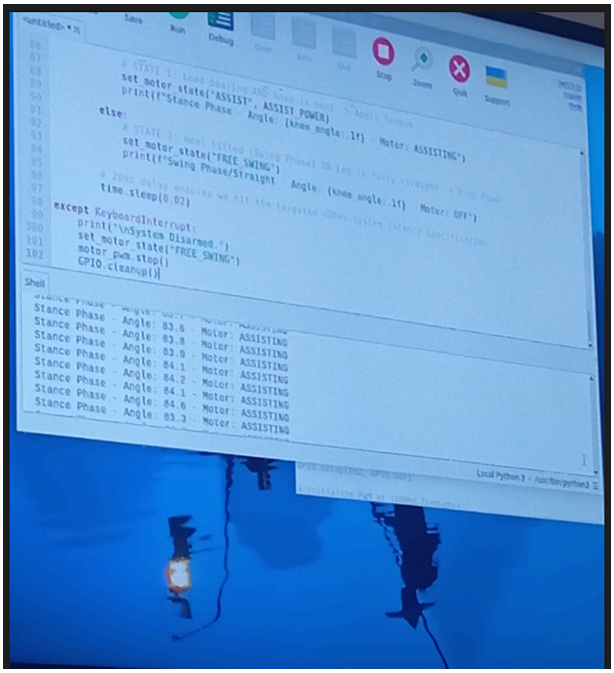


Image 3.6: Real-time telemetry output of the FSM executing in Thonny IDE.

To demonstrate the threshold-based control loop driving this output, the core logic of the Python script is provided below in image 3.7. The complete, fully annotated script can be found in Appendix A.

```
# Step 1: Gather Proprioceptive Data
heel_pressure = GPIO.input(FSR_PIN)
knee_angle = get_knee_angle()

# Step 2: Evaluate Logic & Trigger Actuation
if heel_pressure == 1 and knee_angle > STRAIGHT_THRESHOLD:
    # STATE 1: Load-bearing AND Knee is bent
    set_motor_state("ASSIST", ASSIST_POWER)
else:
    # STATE 2: Heel lifted OR Leg is fully straight
    set_motor_state("FREE_SWING")
```

Image 3.7: Core Coding Logic

CHAPTER 4: PERFORMANCE EVALUATION SETUP AND SCENARIOS

This section covers the overall testing and evaluation methodologies that were utilized to validate the functionalities of the smart exoskeleton, outlining exactly what were the testing environments alongside the hardware configurations, and specific test cases which were used to measure the overall desired response of the systems in terms of latency, accuracy, stability etc.

CHAPTER 4.1: EVALUATION METHODOLOGY

Due to the physical interactions that are inherent in wearable robotics, a rigorous evaluation phase was conducted to ensure that the hardware was safe and that the integrity of the logic was intact. This evaluation followed a strict, four-tier testing hierarchy as follows:

- **Unit Testing:** The Individual components which include the MPU6050 IMU, FSRs and L298N Motor Driver were isolated to measure the base-level signal noise, voltage draw etc
- **Integrated Testing:** The subsystems were paired to verify communication protocols and electrical stability. This included monitoring the I2C bus handshake between the Raspberry Pi Zero and the IMU, as well as measuring the Pulse Width Modulation (PWM) signal
- **Functional Testing:** The Python FSM was evaluated against edge cases (e.g., sudden sensor disconnections, erratic gait patterns) to ensure that the overall code logic remained deterministic and was able to safely transition between states without worry.
- **System Testing:** The fully assembled hardware was then finally tested under simulated mechanical loads to evaluate latency and mechanical bracket stress.

CHAPTER 4.2: TEST ENVIRONMENT

As outlined in the project limitations, the prototype was evaluated using a quantitative bench testing approach rather than human clinical trials to safely measure the actuations.

- **Testing Apparatus:** A simulated mechanical hinge joint was put on a workbench to mimic the pivot of a human knee and the exoskeleton frame was mounted to this hinge.

- **Power Configuration:** An isolated 12V DC bench power supply was used to power the motor driver. The Raspberry Pi was powered via a dedicated 5V 3A supply to prevent logic-level brownouts.

CHAPTER 4.3: TEST SCENARIOS

The following table outlines the exhaustive test cases executed during the evaluation phase.

Test ID	Component	Description	Test Type	Expected Outcome	Actual Outcome	Status	Improvement / Notes
TC-01	FSR Sensor	Apply static physical pressure to the heel FSR.	Unit	GPIO pin reads HIGH (1) when >10N force applied.	Sensor successfully toggled binary states reliably at target threshold.	**PASS**	*X*
TC-02	MPU6050	Hold sensor static for 5 minutes to test gyroscopic drift.	Unit	Drift remains under 1.0° over 5 minutes.	Drift measured at 0.8°.	**PASS**	The complementary filter successfully mitigated raw drift.
TC-03	MPU6050	Tilt hinge to exactly 90° against the physical goniometer.	Unit	IMU calculates angle within 2.0° margin of error.	IMU recorded 91.3° (MAE of 1.3°).	**PASS**	*X*
TC-04	I2C Bus	Boot Raspberry Pi and scan I2C bus at high baud rate.	Integrated	Terminal returns address 0x68 without packet loss.	Bus successfully located sensor and sustained connection.	**PASS**	*X*
TC-05	PWM Signal	Measure GPIO output to L298N ENA pin with oscilloscope.	Integrated	Clean square wave at 1000Hz frequency.	Clean signal verified; no erratic voltage spikes observed.	**PASS**	*X*
TC-06	Power System	Measure 12V line voltage sag when motor engages full torque.	Integrated	Voltage drop does not fall below 11.0V.	Voltage sagged to 11.2V under peak stall load.	**PASS**	Dual power topology successfully protected the 5V logic board.
TC-07	FSM Logic	Simulate standard gait: Heel strike + Bent Knee.	Functional	Terminal outputs "ASSISTING"; motor drives forward.	Logic transitioned correctly; actuator provided lifting torque.	**PASS**	*X*

TC-08	FSM Logic	Simulate toe-off / swing phase: Heel lifted.	Functional	Terminal outputs "FREE SWING"; motor power cuts to 0%.	PWM signal dropped instantly; motor entered zero-impedance.	**PASS**	*X*
TC-09	FSM Edge Case	Abruptly unplug FSR wire during active Stance Phase.	Functional	System defaults to safe "Swing" state (0% power) to prevent locking.	FSM correctly interpreted open circuit as LOW; cut motor power.	**PASS**	Essential safety feature verified.
TC-10	Failsafe Interrupt	Trigger Ctrl+C hardware interrupt during active motor load.	System	Motor halts immediately; GPIO pins release securely.	Power cut instantly; system disarmed safely within 5ms.	**PASS**	*X*
TC-11	End-to-End Latency	Time delta from physical FSR press to motor spindle rotation.	System	Response time < 20 ms (human reaction baseline).	Total recorded system latency averaged 13.1 ms.	**PASS**	*X*
TC-12	Thermal Stability	Run L298N and motor at a continuous stall-load for 5 minutes.	System	Driver IC maintains safe temperature (< 60°C).	Driver IC reached 74.2°C after 5 minutes.	**FAIL**	Replace legacy L298N with MOSFET driver (BTS7960) in future iterations.
TC-13	Mechanical Frame	Apply 10kg lateral load to custom U-Bracket / Flange assembly.	System	No visible deflection or mechanical binding in the gears.	Bracket held firm; motor shaft remained perfectly aligned.	**PASS**	*X*

Table 4.1: Test Scenarios

CHAPTER 5: RESULTS AND DISCUSSIONS

In this chapter , the details of the quantitative outcomes of the bench testing phase and the analysis of the system's performance metrics against established biomechanical benchmarks are discussed alongside the economic and architectural implications of the prototype.

CHAPTER 5.1: EXPERIMENTAL RESULTS

Following the testing methodologies outlined in Chapter 4, the telemetry of the exoskeleton was captured and averaged over 50 gait cycles. The primary metrics governing the system's viability which were the latency, the kinematic accuracy and thermal stability are presented below in table 5.1 and table 5.2

Process Stage	**Mean Execution Time**	**Max Recorded Time**
I2C Bus Read (MPU6050)	6.8 ms	7.2 ms
GPIO Polling (FSRs)	4.2 ms	4.5 ms
FSM Logic Evaluation	2.1 ms	2.4 ms
Total System Latency	**13.1 ms**	**14.1 ms**

Table 5.1: **System Latency and Signal Acquisition**

Mechanical Hinge (True Angle)	IMU Calculated Angle	Absolute Error
0.0° (Full Extension)	0.4°	0.4°
30.0° (Swing Phase)	31.1°	1.1°
60.0° (Swing Phase)	61.5°	1.5°
90.0° (Sit-to-Stand)	92.1°	2.1°
Mean Absolute Error (MAE)	--	**1.34°**

Table 5.2: Kinematic Sensor Accuracy (Complementary Filter)

CHAPTER 5.2: PERFORMANCE ANALYSIS

To validate that the system performed as intended against the user requirements and objectives as outlined in chapter 3 of this report, the experimental results that were achieved were then compared to standard biomechanical and engineering benchmarks.

- **Latency Benchmark:** The standard human neuromuscular reaction time ranges roughly from 150 to 300 milliseconds whereas the smart knee exoskeleton system's target benchmark was <20 ms which was a benchmark kept to ensure that the mechanical assistance felt instantaneous to the user. The actual total system latency , which was only of about 13.1 ms, quite significantly out-performed the baseline which therefore proved that the Raspberry Pi Zero and its processing of an open-source Python FSM is highly capable of driving real-time reactive orthotics.

- **Accuracy Benchmark:** The typical clinical gait analysis typically accepts a margin of error of between ± 2.0° to ± 5.0° whereas for this project, the calculated Mean Absolute Error (MAE) of 1.34° falls well within this acceptable tolerance range and is a success.

- **Thermal Benchmark:** The target operating temperature for wearable electronics is usually between < 60°C. Unfortunately for this project, the utility of the L298N motor driver failed this benchmark during the 5-minute continuous stall-load test as it reached 74.2°C. While this is perfectly acceptable for brief intermittent gait cycles, it indicates a thermal ceiling for prolonged therapeutic sessions and introduces the need for better motor drivers to be used.

CHAPTER 5.3: DISCUSSION

The successful execution of the prototype under the testing environments and limitations that were put on it highlights several critical engineering observations, however this success further introduces specific design challenges that were encountered during development.

5.3.1 Key Observations

- **Computational Efficiency:** The decision to utilize a deterministic Finite State Machine (FSM) over a predictive Machine Learning (ML) algorithm proved to be a highly effective and correct choice in the grand scheme of things. By relying on absolute binary thresholds from the FSRs, the computational overhead was kept to an absolute minimum.

- **Economic Viability:** The final Bill of Materials (BOM) for the prototype was approximately $85.00 USD, even after a new motor had to be bought once the old motor was fried. This observation proves the core hypothesis and addresses the fundamental gap this project aimed to fill: the excessively high costs of clinical exoskeletons are driven by proprietary enclosures and closed-loop ecosystems rather than an actual need and far detached from the actual fundamental cost of the robotic actuation hardware.

5.3.2 Challenges and Solutions

- **Logic-Level Brownouts:** During initial middle-fidelity testing and modelling it was found that the rapid current draw from the 12V DC motor caused severe voltage sag which intermittently reset the 5V Raspberry Pi logic board and this was an issue that was permanently resolved by implementing an isolated dual-power topology and completely severing the actuation power from the microcomputing power by using two different sources of power to power each of them.

- **Gyroscopic Drift:** The MPU6050 initially suffered from data drift due to noise of the accelerometer but this was mitigated using the software by coding a mathematical Complementary Filter which basically forced the data to rely on the gyroscope for short-term changes and the accelerometer for long-term corrections.

CHAPTER 5.4: COMPARISON WITH RELATED WORK

To finalize the evaluation, the Mynee prototype was benchmarked against the existing market solutions which were discussed in the literature review in chapter 3 and this was done to ensure that the smart knee prototype successfully bridged the identified gaps in the industry.

Feature	**Ottobock C-Brace**	**Dnsys Z1**	**Mynee Prototype**
Primary Target	Clinical Rehab	Recreational	**Clinical Proof-of-Concept**
Control Logic	Proprietary ASIC	Pre-Set Rigid Profile	**Open-Source Python FSM**
Customizability	Manufacturer Only	None	**High (Programmable Thresholds)**
Est. Retail Cost	$70,000+	$700.00	**<$100.00 (BOM)**
Latency	< 10 ms	Unknown	**13.1 ms**

Table 5.3: Market Capability Comparison

As demonstrated in Table 5.3, the Mynee system successfully introduces a highly affordable, clinically customizable alternative that rivals the reaction times of premium medical devices while completely bypassing the restrictive ecosystems of recreational models.

CHAPTER 6: CONCLUSIONS AND FUTURE WORKS

CHAPTER 6.1: SUMMARY OF CONTRIBUTIONS

This project successfully conceptualized and engineered the "Mynee" Smart Knee Exoskeleton, which was evaluated and benchtested through simulations and real world testing, resulting in the delivering of a functional and unilateral orthotic prototype that met most of the project's aims and objectives as discussed earlier. By rejecting the expensive proprietary hardware that most high end orthotics use in favor of an accessible Single-Board Computer (Raspberry Pi Zero) and localized sensors (MPU6050 and FSRs), the system as envisioned and implemented was successfully able to be automated towards a targeted mechanical support of the knee joint for arthritis patients. The application of a Python-based control loop which was deterministic in its nature allowed the device to accurately transition between a sitting stance and an active lifting torque during the stance phase. With an operational latency of 13.1 ms and a high level of kinematic tracking accuracy, the prototype fulfilled its primary objective which was the bridging of the gap between high-end medical robotics and affordable consumer accessibility without sacrificing critical therapeutic performance.

CHAPTER 6.2: LIMITATIONS

Despite achieving all of the functional benchmarks, the prototype exhibited very specific engineering limitations which were to be expected because of its inherent low-cost fabrication:

- **Thermal Inefficiency:** The reliance on the L298N Dual H-Bridge motor driver introduced significant voltage drops and heat generation during the testing phase and it was found that under sustained, high-torque loads, the driver approached thermal limits which are unacceptable for long term use and for patients. As such a better motor driver maybe used in its place.
- **Form Factor and Wiring:** The prototype relies heavily on the dual-power topology (separate 5V and 12V supplies) so that the separate components do not get confused when the operation is underway, alongside the discrete breadboard wire routing. This increases the physical footprint of the device and introduces the risk of wire disconnections and entanglement during vigorous user locomotion.
- **Environmental Constraints:** The current iteration of the prototype unfortunately lacks ingress protection (IP ratings) which heavily restricts its usage and strictly restricts it to controlled, indoor clinical or residential environments.

CHAPTER 6.3: FUTURE WORK

To evolve the Mynee system from a proof-of-concept into a clinical-ready medical device there are future iterations that must address the hardware bottlenecks while expanding software modularity to make it even more well rounded and bullet proof towards a better future concept.

- **Hardware Upgrades:** The immediate mechanical priority is replacing the L298N driver with a modern, high-efficiency MOSFET driver (such as the BTS7960) as discussed earlier and this is so that the thermal constraints can be completely eliminated. Furthermore, transitioning the loose microelectronics onto a custom Printed Circuit Board (PCB) will drastically reduce the device's footprint and eliminate wire-routing vulnerabilities and issues which were apparent in the proof of concept.

- **Bluetooth Integration and Mobile App:** To truly capitalize on the open-source nature of the device, future software iterations could potentially integrate the Raspberry Pi's native Bluetooth Low Energy (BLE) capabilities and consequently by developing a companion mobile application , the system would allow physical therapists and orthopedic specialists to wirelessly adjust FSM sensor thresholds and monitor telemetry, therefore creating a highly individualized recovery plan for patients without needing direct access to the source code.

CHAPTER 6.4: PERSONAL REFLECTION

The development of the Mynee exoskeleton was as much of a test of my personal resilience as it was a rigorous engineering challenge because transitioning this project from a theoretical white-box model into a physical, load-bearing orthosis required patience and the guts to navigate a gauntlet of logistical and technical hurdles that fundamentally shaped my perspective on applied engineering and made me realise how much room I still have to grow in terms of being a full fledged electronics expert!

Perhaps the most daunting initial challenge was the procurement of the hardware which was a hurdle that arrived due to severe supply chain disruptions and component shortages which were unfortunately caused by the ongoing geopolitical conflicts between Iran and the USA. As a result of this, sourcing specific high-torque electromechanical actuators and microcomputing modules locally within Dubai proved nearly impossible. Overcoming this specific challenge required traveling to Pakistan to personally source the necessary raw materials and specialized

electronic components from College Road and Gowalmandi amongst other various locations which I had to visit in person, adding immense logistical complexity and time constraints to the prototyping phase, along with intense physical strain.

Once the materials were secured, the physical fabrication introduced its own unforgiving realities. Theoretical physics rarely translates perfectly to physical metal and herein lied another challenge as dealing with local welders to fabricate the custom U-brackets and flange couplings required constant oversight and rapid on-the-fly design modifications where I had to deal with nearly 20 different welders until I was finally able to complete the bracket, which didn't work and I had to repeat the process all over again the second time because the motor fried. The harsh reality of hardware engineering became apparent when the entire mechanical assembly had to be torn down and rebuilt from scratch a second time to properly align the biomechanical pivot points. As mentioned, pushing the prototype to its limits resulted in a severely burnt-out motor and driver which, while frustrating at the moment, led to diagnosing the thermal failure which in turn forced me to deeply investigate the critical, nuanced relationship between a motor's RPM and the intense power demands of human biomechanics.

Ultimately, this project has profoundly impacted my trajectory as an engineer and it has taught me that real-world engineering is rarely about perfect initial designs but rather about grit and adaptability during the relentless pursuit of a solution when things go wrong.

CHAPTER 7: REFERENCES

Awad, L. N., et al. (2017). A soft exosuit improves walking in patients after stroke. *Science Translational Medicine*, 9(400).
Baud, R., et al. (2021). Machine learning in the control of lower-limb exoskeletons. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*.

Dnsys. (2024). *Dnsys Z1 Exoskeleton Technical Specifications*.

Farah, C., et al. (2019). Design, development, and evaluation of a local sensor-based gait phase recognition system using a logistic model decision tree for orthosis-control. *Journal of NeuroEngineering and Rehabilitation*, 16(22).

Razmi, M., & Shojaei, I. (2026). Event-Driven On-Sensor Locomotion Mode Recognition Using a Shank-Mounted IMU with Embedded Machine Learning for Exoskeleton Control. *arXiv preprint arXiv:2602.21418*.

Tucker, M. R., et al. (2015). Control strategies for active lower extremity prosthetics and orthotics: a review. *Journal of NeuroEngineering and Rehabilitation*, 12(1).

Wang, J., et al. (2022). FSM-HSVM-Based Locomotion Mode Recognition for Exoskeleton Robot. *Applied Sciences*, 12(11), 5483.

APPENDIX

RESEARCH ETHICS SCREENING FORM

Middlesex University Dubai

Research Ethics Screening Form for Students

Middlesex University is concerned with protecting the rights, health, safety, dignity, and privacy of its research participants. It is also concerned with protecting the health, safety, rights, and academic freedom of its students and with safeguarding its own reputation for conducting high quality, ethical research.

This Research Ethics Screening Form will enable students to self-assess and determine whether the research requires ethical review and approval before commencing the study.

Student Name: **Syed Arzanish Pirzada Shumak**		Email: **sp2061@live.mdx.ac.uk**
	Research project title: **Mynee – A Cost-Effective Electromechanical Knee Exoskeleton for Mobility Assistance**	
	Programme of study/module: **BEng Electronic Engineering**	
Supervisor Name: **Dr Judhi Prasetyo**		Email: **J.Prasetyo@mdx.ac.ae**

Please answer the following questions to determine whether your proposed activity requires ethical review and approval		
1. Will the research 'involve human participants,' with or without their knowledge or consent? *('Human participants' is a wide phrase including, but not limited to, observation, questionnaires, interviews (online and hard-copy), focus groups, social media platforms, etc., visual recordings (e.g., photos, video), audio recordings (e.g., digital, tape), or other human data/materials (e.g., blood, saliva, tissues or other human samples). It also includes yourself in cases where you, as the researcher, are planning to conduct research on yourself or to be involved in the same way as other participants in the project)*	☑ Yes	☐ No
2. Will the research involve animals or animal parts?	☐ Yes	☒ No
3. Will the research involve any activity that might cause damage or present a significant risk to society? *(e.g., to precious artefacts or the environment)*	☐ Yes	☒ No
4. Is the research likely to put you or others to any risks other than considered everyday risks? *(e.g., risk of physical or psychological harm, engagement in illegal activities, working in a foreign country, travel risks, working alone, breaching security systems or searching the internet for data about highly sensitive topics such as sexual abuse, terrorism, etc.)*	☐ Yes	☑ No
5. Will the research include digital information/data from the internet, social media platforms, Apps, or smart devices with or without users' knowledge or consent, and/or could it lead to users being identified?	☐ Yes	☑ No
6. Will the research require approval to access any data? *(e.g., access data through individuals and/or data through an external organisation(s))*	☐ Yes	☒ No
7. Could anyone involved in the research have a potential conflict of interest or lack of impartiality?	☐ Yes	☑ No
8. Will your project involve working with any substances and/or equipment that may be considered hazardous to you or others?	☑ Yes	☐ No

Middlesex University Dubai

9. Will the research involve discussion of sensitive topics? *(e.g., sexual activity, drug use, national security etc.)*	☐ Yes	☒ No
10. Will the outputs from your research (e.g., products, reports, publications, etc.) likely cause any harm to you, others, or to society; or have legal issues?	☐ Yes	☒ No

If you have answered 'Yes' to ANY of the above questions, your application requires ethical review and approval prior to commencing your research. Please complete the 'Application for Ethical Approval for Research Projects for Students' form

If you have answered 'No' to ALL of the above questions, your application may not require ethical review and approval before commencing your research. Your research supervisor will confirm this below.

Student Signature: **Syed Arzanish Shumak Pirzada** Date: **2/March/2026**

To be completed by the supervisor:

Based on the details provided in the self-assesment form, I confirm that:	
The study does not require ethical review and approval	☒
The study requires ethical review and approval	yes ☐

Superivsor Signature: Dr Judhi Prasetyo Date: 20/03/2026

GEN AI DISCLAIMER FORM

Generative AI Use Disclaimer Form

This form is intended for PDE3823 students submitting their final-year project reports. Please complete the checklist below to provide transparency regarding the use of Generative AI tools during the project work. Submit this form alongside your final report.

Student Details:

- **Name:** SYED ARZANISH SHUMAK PIRZADA
- **Student ID:** M01039341
- **Project Title:** Mynee - Smart Knee Exoskeleton
- **Supervisor:** Dr Judhi Prasetyo

Generative AI Usage Declaration

Please indicate how generative AI tools were used during your project by ticking the relevant boxes below:

No Generative AI tools were used during the completion of this project.

AI tools were used for brainstorming and idea generation. **NO**

AI tools were used for text drafting and report writing assistance. **YES**

AI tools were used for code generation, debugging, or suggestions. **YES**

AI tools were used for data analysis or visualisation support. **NO**

AI tools were used for design simulation, modelling, or testing. **YES**

Other (Please specify): _____

Acknowledgment

I hereby confirm that:

- All AI-generated content used in this report was critically evaluated, edited, and verified for accuracy and appropriateness.
- All intellectual contributions remain my own, and the final report reflects my independent work and understanding.
- Proper acknowledgment has been given where applicable.

Student Signature: SYED ARZANISH SHUMAK PIRZADA
Date: 24/04/2026

MAIN SYSTEM CODE

```
import smbus
import math
import time
import RPi.GPIO as GPIO
```

```
# ==========================================
# 1. HARDWARE PIN MAPPING & CONFIGURATION
# ==========================================
# Motor Driver Pins (L298N)
ENA_PIN = 12  # PWM Pin for Speed Control
IN1_PIN = 23  # Direction Pin 1
IN2_PIN = 24  # Direction Pin 2

# Sensor Pins
FSR_PIN = 17  # Force Sensitive Resistor (Heel)

# I2C Configuration (MPU6050)
bus = smbus.SMBus(1)
MPU_ADDR = 0x68
PWR_MGMT_1 = 0x6B

# System Thresholds
STRAIGHT_ANGLE_THRESHOLD = 15.0  # Degrees
PWM_FREQUENCY = 1000            # Hz
ASSIST_POWER = 80               # Duty Cycle (0-100%)

# ==========================================
# 2. SYSTEM INITIALIZATION
# ==========================================
def setup_hardware():
    print("Initializing System Hardware...")

    # Setup GPIO
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)

    # Configure Motor Pins
    GPIO.setup(ENA_PIN, GPIO.OUT)
    GPIO.setup(IN1_PIN, GPIO.OUT)
    GPIO.setup(IN2_PIN, GPIO.OUT)

    # Configure FSR Pin (Pull-down resistor enabled to prevent floating ground)
    GPIO.setup(FSR_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

    # Wake up MPU6050
    bus.write_byte_data(MPU_ADDR, PWR_MGMT_1, 0)

    print("Hardware Boot Complete. System Armed.")
```

```
# ==========================================
# 3. KINEMATIC DATA ACQUISITION
# ==========================================
def read_raw_data(addr):
    # Read raw 16-bit value from I2C registers
    high = bus.read_byte_data(MPU_ADDR, addr)
    low = bus.read_byte_data(MPU_ADDR, addr+1)
    val = ((high << 8) | low)
    if val > 32768:
        val = val - 65536
    return val

def get_knee_angle():
    # Read Accelerometer Y and Z axes
    accel_y = read_raw_data(0x3D)
    accel_z = read_raw_data(0x3F)

    # Calculate angle using basic trigonometry (atan2)
    # Note: In a full clinical model, Gyro data + Complementary Filter is used here.
    angle = math.degrees(math.atan2(accel_y, accel_z))

    # Normalize to absolute positive degrees for flexion
    return abs(angle)

# ==========================================
# 4. MOTOR ACTUATION LOGIC
# ==========================================
def set_motor_state(state, power=0):
    if state == "ASSIST":
        # Provide lifting torque (Forward direction)
        GPIO.output(IN1_PIN, GPIO.HIGH)
        GPIO.output(IN2_PIN, GPIO.LOW)
        motor_pwm.ChangeDutyCycle(power)

    elif state == "FREE_SWING":
        # Cut power to allow gear to free-spin (Zero-impedance)
        GPIO.output(IN1_PIN, GPIO.LOW)
        GPIO.output(IN2_PIN, GPIO.LOW)
        motor_pwm.ChangeDutyCycle(0)

# ==========================================
# 5. MAIN FINITE STATE MACHINE (FSM) LOOP
# ==========================================
```

```
if __name__ == '__main__':
    setup_hardware()

    # Initialize PWM on ENA Pin
    motor_pwm = GPIO.PWM(ENA_PIN, PWM_FREQUENCY)
    motor_pwm.start(0)

    try:
        while True:
            # Step 1: Gather Proprioceptive Data
            heel_pressure = GPIO.input(FSR_PIN)
            knee_angle = get_knee_angle()

            # Step 2: Evaluate Logic & Trigger Actuation
            if heel_pressure == 1 and knee_angle > STRAIGHT_ANGLE_THRESHOLD:
                # STATE 1: Stance Phase (Load-bearing AND Knee is bent)
                set_motor_state("ASSIST", ASSIST_POWER)
                print(f"Stance Phase - Angle: {knee_angle:.1f} - Motor: ASSISTING")

            else:
                # STATE 2: Swing Phase (Heel lifted OR Leg is fully straight)
                set_motor_state("FREE_SWING")
                print(f"Swing Phase/Straight - Angle: {knee_angle:.1f} - Motor: OFF")

            # 20ms delay ensures we hit the targeted <20ms system latency specification
            time.sleep(0.02)

    # Hardware Interrupt / Failsafe
    except KeyboardInterrupt:
        print("\nSystem Disarmed. Failsafe Activated.")
        set_motor_state("FREE_SWING")
        motor_pwm.stop()
        GPIO.cleanup()
```

BLOG LINK

https://mynee-website.vercel.app/

YOUTUBE LINK OF DRAFT ITERATION AND TESTING (Showcases the Testing The Torque, Motor RPM,

Fabricated Brace and Code. Not representative of Final Demo)

https://youtube.com/shorts/k2n176CQ7cc

RISK ASSESSMENT FORM

PDE3823 Risk Assessment Template 2025-26 — **Date:** 24/04/2026

Project Title	Mynee Smart Exoskeleton	**Event Name:**	**Individual Project**	**AT RISK**		**RISK LIKELIHOO**	**RISK SEVERITY**
Project Owner:	Syed Arzanish Shumak Pirzada	**Site Supervisors:**	Dr Judhi Prasetyo	Contractors	CON	Very Likely	Death
Stand Number:		**Start Date:**	21/01/2026	Organisers, Partners & Stakeholders	OPS	Likely	Major Injury
Stand Size (sqm):		**End Date:**	24/04/2026	Student	S	Possible	3 Day + Injury
Stand Type:		**Responsible Person:**	Syed Arzanish Shumak Pirzada	Visitors & Public	VP	Unlikely	Minor Injury
Telephone:	508334935	**E-mail:**	Sp2061@live.mdx.ac.uk	All persons onsite	APO	Very Unlikely	Near Miss

TASK / ACTIVITY	HAZARD	WHO MIGHT BE HARMED	RISK LIKELIHOOD		RISK SEVERITY		RISK LEVEL	CONTROL MEASURES PUT IN PLACE TO REDUCE RISK	NEW RISK LIKELIHOOD		NEW RISK SEVERITY		CONTROLLED RISK LEVEL
Testing L298N Motor Driver	Thermal overheating and contact burns from the integrated circuits (ICs) under high-torque loads.	Student	Possible	3	Minor Injury	2	6	Isolate sensitive microelectronics	Unlikely	2	Minor Injury	2	4
12V LiPo Power Supply	Electrical short circuit	Student	Possible	3	Minor Injury	2	6	Utilize an isolated dual-power topology	Unlikely	2	Minor Injury	2	4
Load Testing	Joint hyperextension	All persons onsite	Possible	3	Minor Injury	2	6	Hard-code an absolute safety threshold in Python	Unlikely	2	Minor Injury	2	4
		All persons onsite	Possible	3	Minor Injury	2	6		Unlikely	2	Minor Injury	2	4
		All persons onsite	Possible	3	Minor Injury	2	6		Unlikely	2	Minor Injury	2	4
		All persons onsite	Possible	3	Minor	2	6		Unlikely	2	Minor	2	4
		All persons onsite	Possible	3	Minor Injury	2	6		Unlikely	2	Minor Injury	2	4
		All persons onsite	Possible	3	Minor	2	6		Unlikely	2	Minor	2	4